

MODULE 2

ADVANCED DATA STRUCTURES

AND

GRAPH ALGORITHMS.

- Self Balancing Tree

- AVL Tree

- Insertion

- Deletion

- Rotation

- Disjoint Sets

- Disjoint set operations

- Union and find opern.

- BFS Traversal + Analysis

- DFS Traversal + Analysis

- Strongly Connected Components

- Topological Sorting.

AVL TREE - $O(\lg n)$

29

An AVL tree is a binary search tree that is height balanced.

For each node x , the heights of the left and right subtrees of x differ by at most 1.

To implement an AVL tree, we maintain an extra field in each node: $h[x]$ is the height of node x .

Definition

* An empty binary tree is an AVL tree.

* If B is the non-empty binary tree with B_L and B_R are its left and right subtrees then B is an AVL tree if and only if

$\Rightarrow B_L$ and B_R are AVL trees, and

$\Rightarrow |h_L - h_R| \leq 1$, where h_L and h_R

are the heights of B_L and B_R

subtrees respectively.

Balance Factor

To implement an AVL tree each node must contain a balance factor, which indicates its status of balance relative to its subtrees.

If balance is defined by

$$\text{Height}(\text{left subtree}) - \text{Height}(\text{right subtree})$$

then the balance factor can have values of $\{-1, 0, 1\}$.

If a node has balance factor 1, then that means left subtree is at greater height than the right subtree, then it is said to be left balanced.

If a node has balance factor -1, then that means right subtree is at greater height than the left

subtree then it is said to be right balanced. (3)

For balance factor 0, the tree is said to be completely balanced.

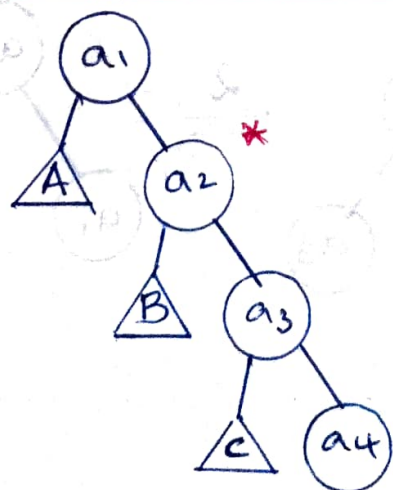
Rotations

Insertion into a tree may cause an unbalanced tree.

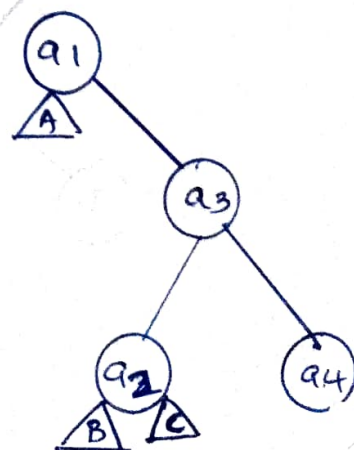
Rebalancing of the tree is performed by performing 4 kinds of rotations:

- Left Left [LL] Rotation
- Right Right [RR] Rotation
- Left Right [LR] Rotation
- Right Left [RL] Rotation.

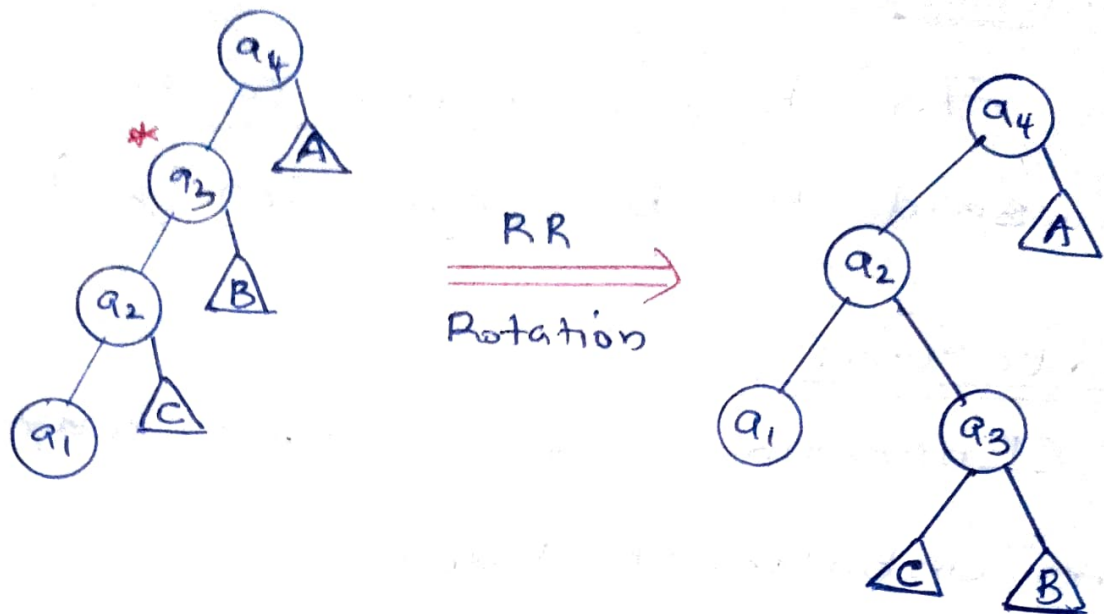
Left-Left Rotation



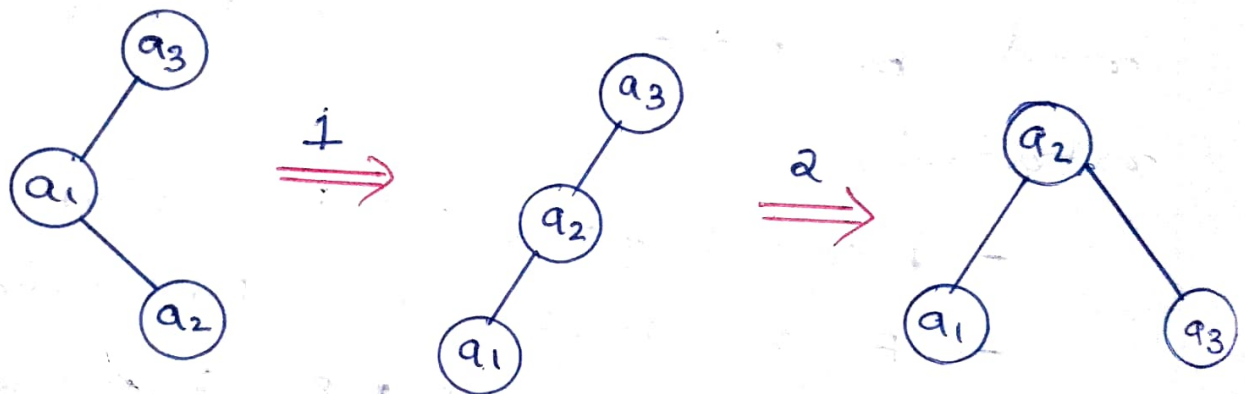
LL
Rotation



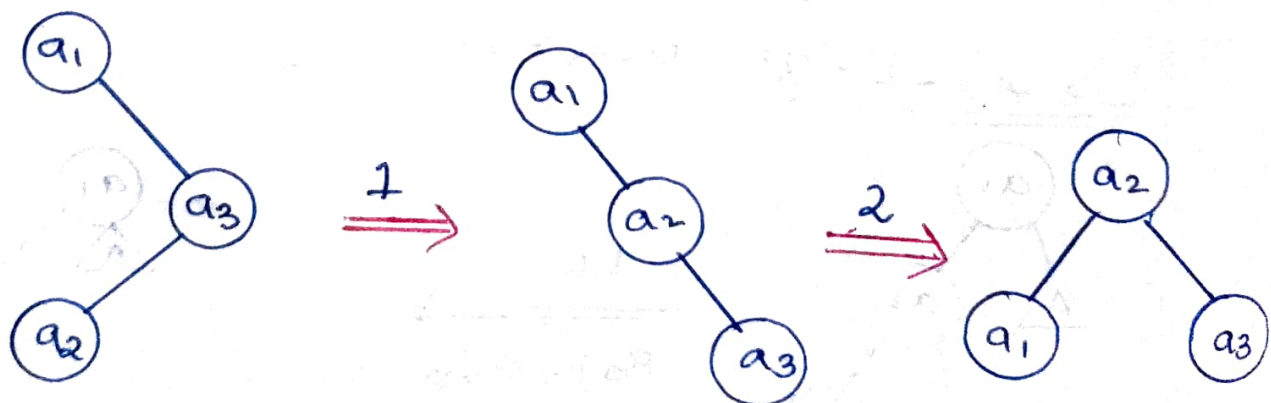
Right-Right Rotation



Left-Right Rotation

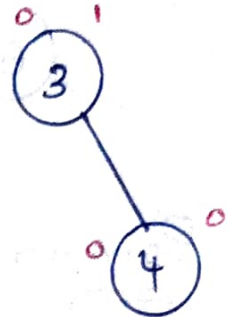
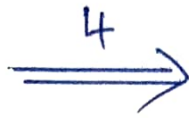


Right-Left Rotation

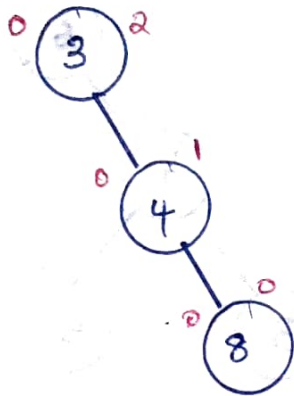


Example Construct an AVL tree from the given set of values. {3, 4, 8, 6, 7, 5, 2, 1} (33)

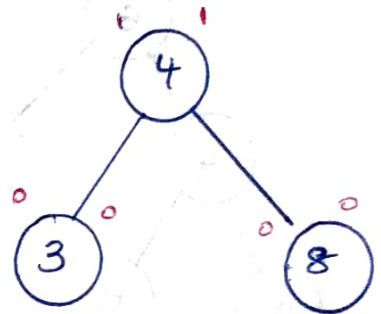
Insert 3



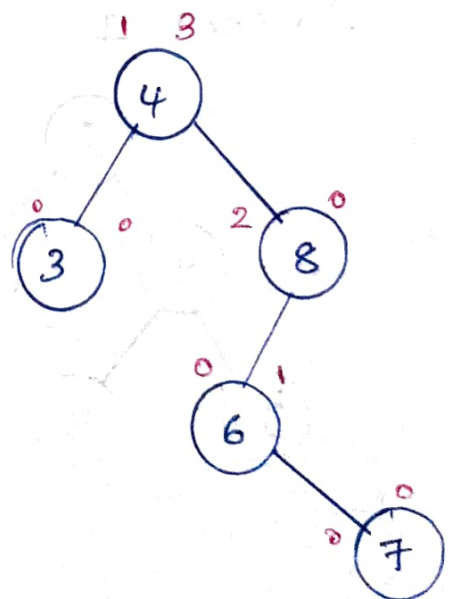
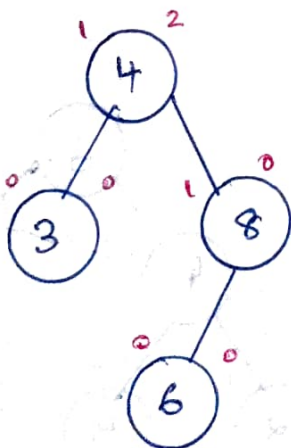
Insert 8



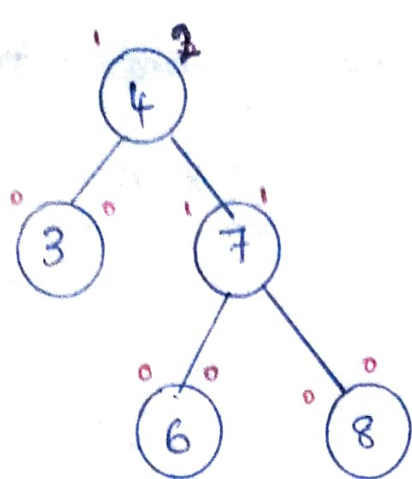
LL
Rotation



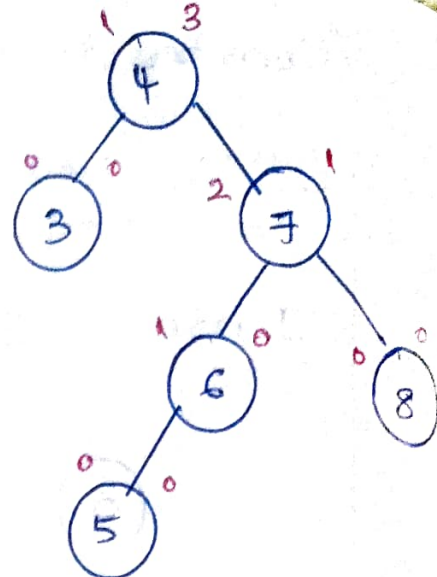
Insert 6



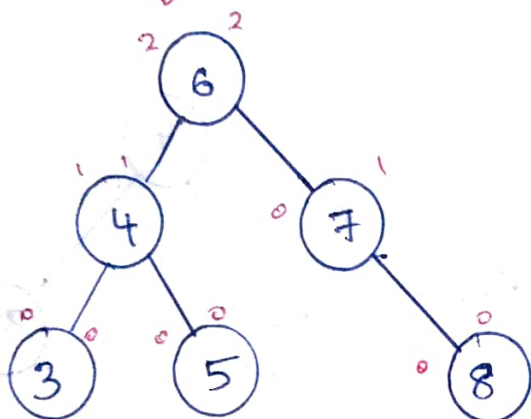
Perform L.R rotation



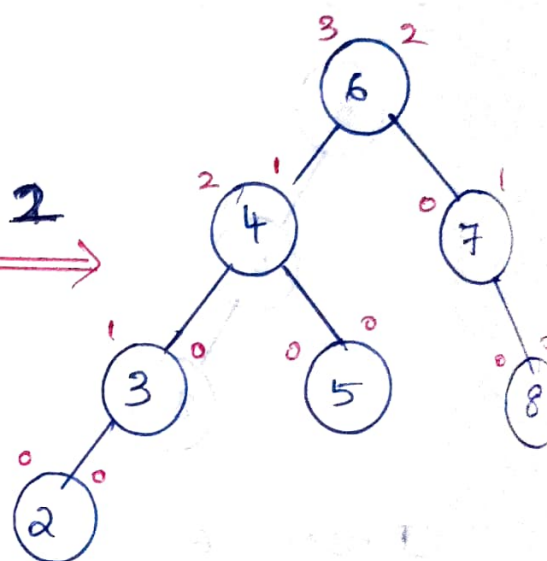
5



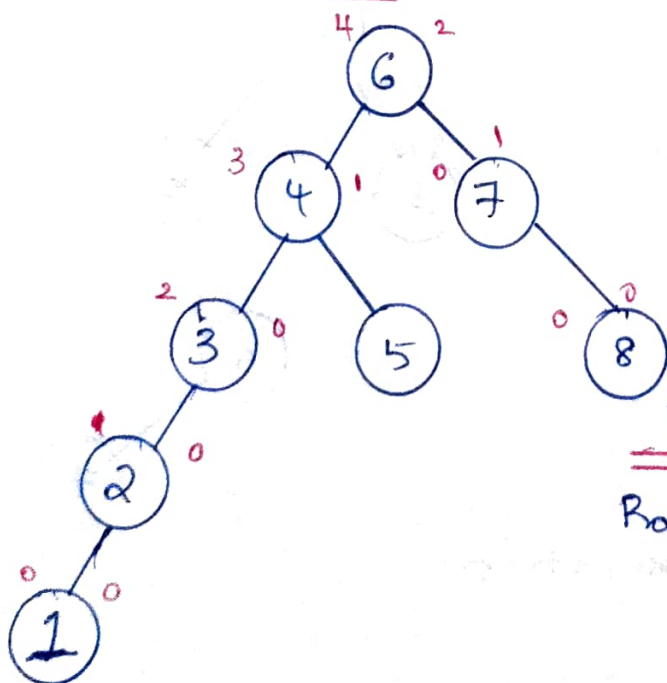
Perform RL rotation



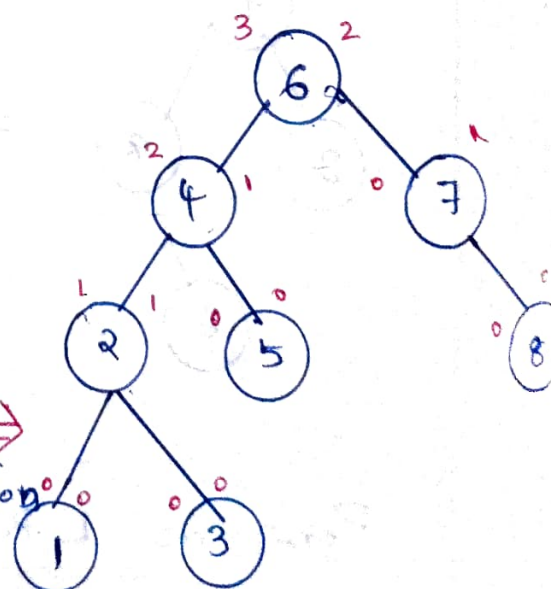
2



Insert 1



RR
Rotation



AVL TREE DELETION

AVL tree deletion is performed in the same way as it is performed in a binary search tree.

There are 3 possible cases to consider. They are

→ The node that is to be deleted here

1 ⇒ No children

2 ⇒ One child

3 ⇒ 2 children [left & Right]

Case 1:

If the node to be deleted is having no child, [or if it is leaf node] it is simply deleted.

Case 2:

If it have one child, then the node is replaced by its only child

Case 3:

If it have both left and right child, then it can be replaced

neither by its

⇒ inorder successor [Smallest node in the right subtree]

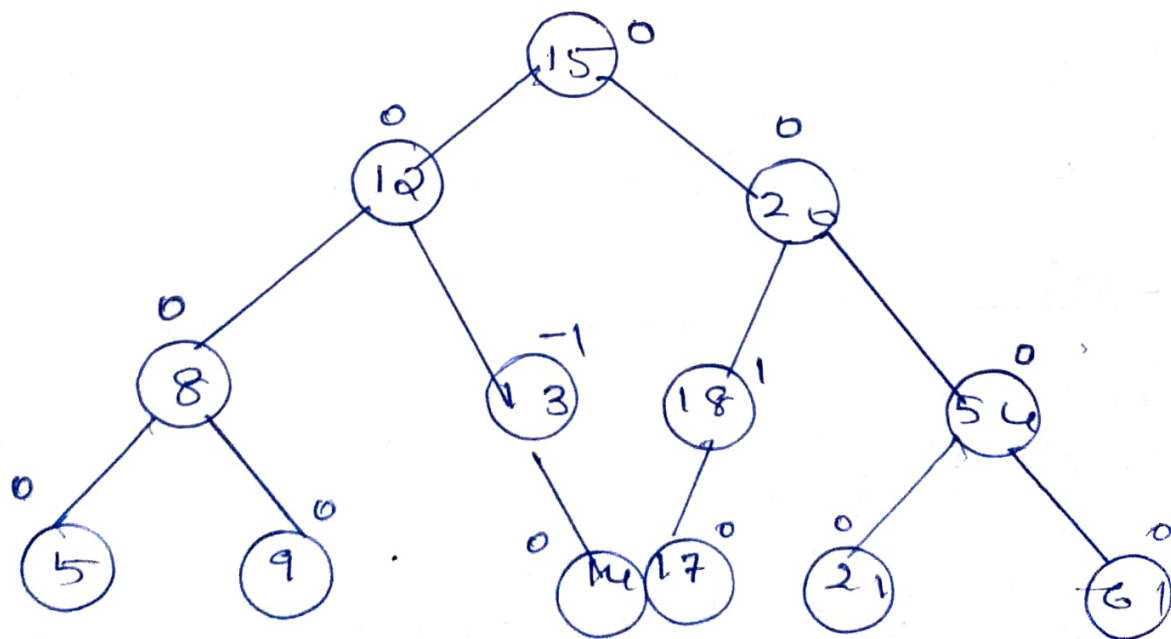
or

⇒ inorder predecessor [Largest node in the left subtree].

After deletion, we have to check the balance of the tree and if there is any variation occurred, we have to perform rotations accordingly.

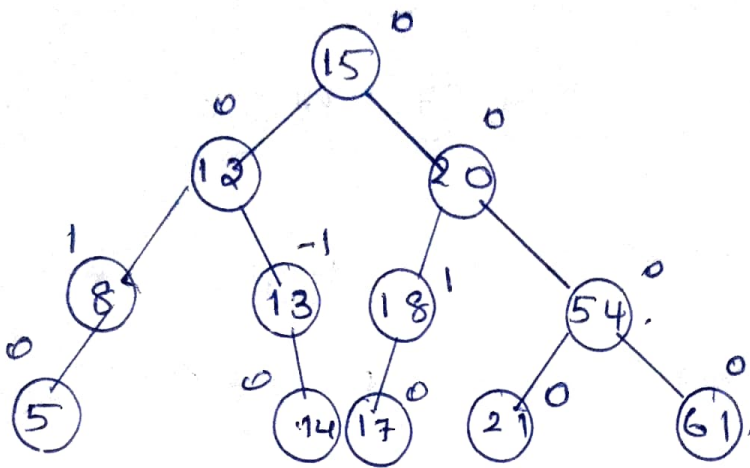
Example

Delete node 9, 8, 12, 15, 18 from the given AVL tree.



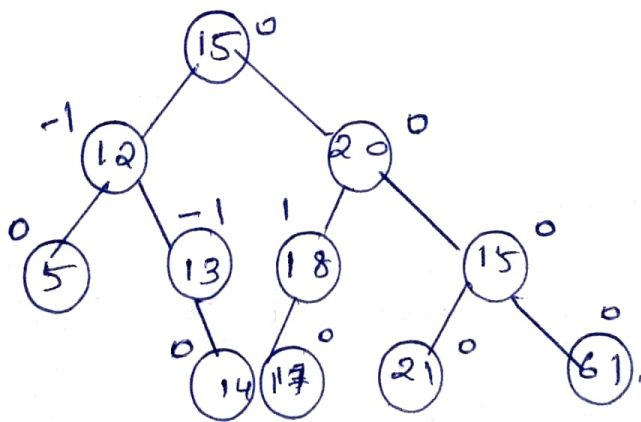
Delete 9

9 is a leaf node, so we simply delete it.



Delete 8

8 is having one child, so after deletion 8 is replaced by 5.

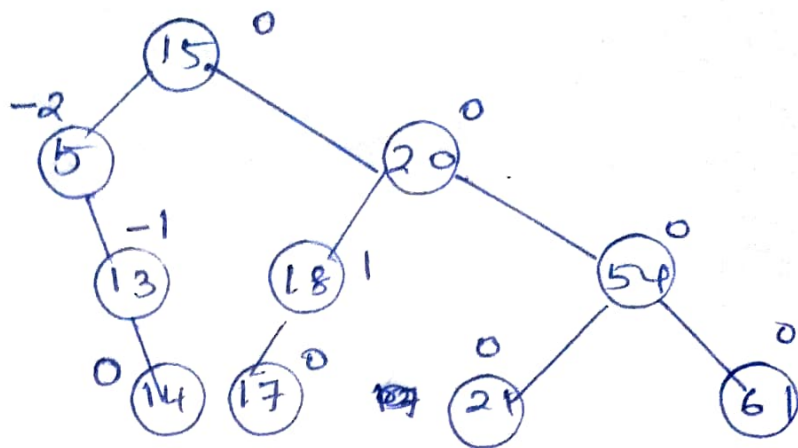


Delete 12

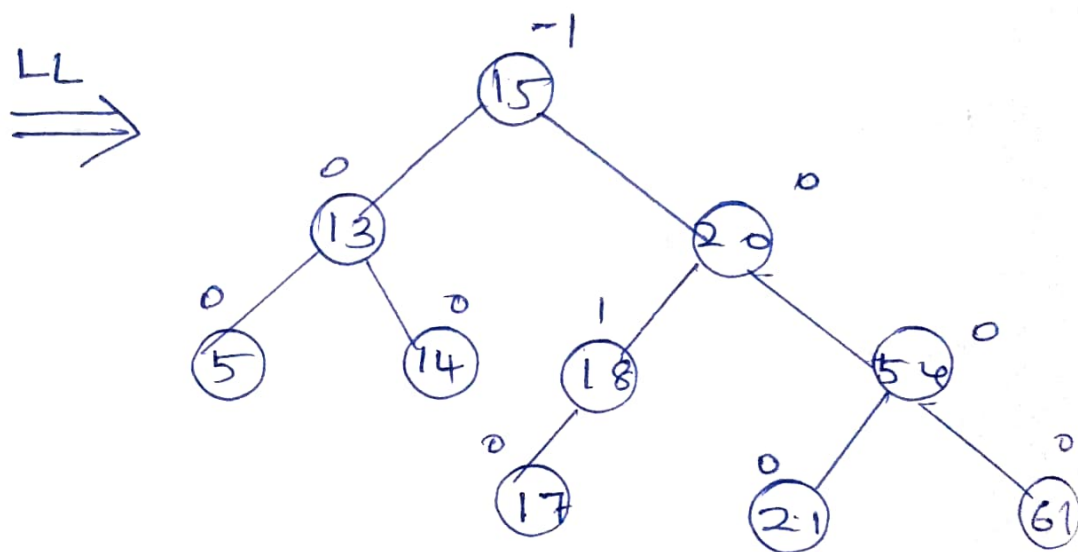
12 have both left and right child.

So either replace 12 by its inorder

Successor [13] or by its inorder predecessor [5].



Variation in b.f. occurs at 5.
So rotation is performed.

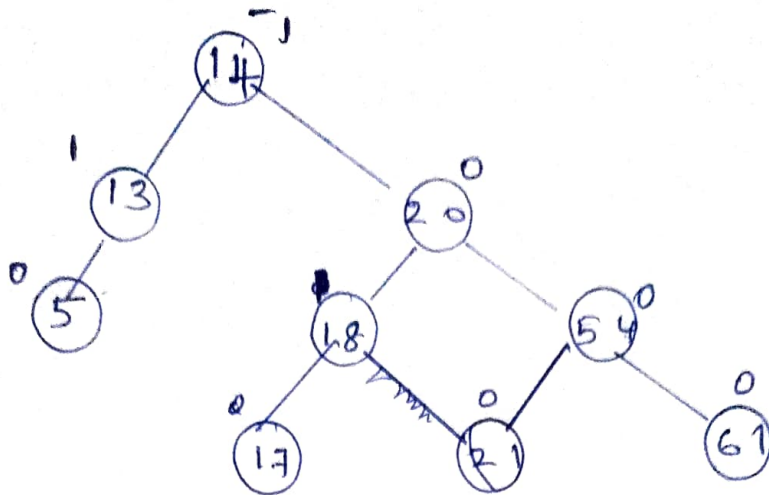


Delete 15

15 have both left and right children, either

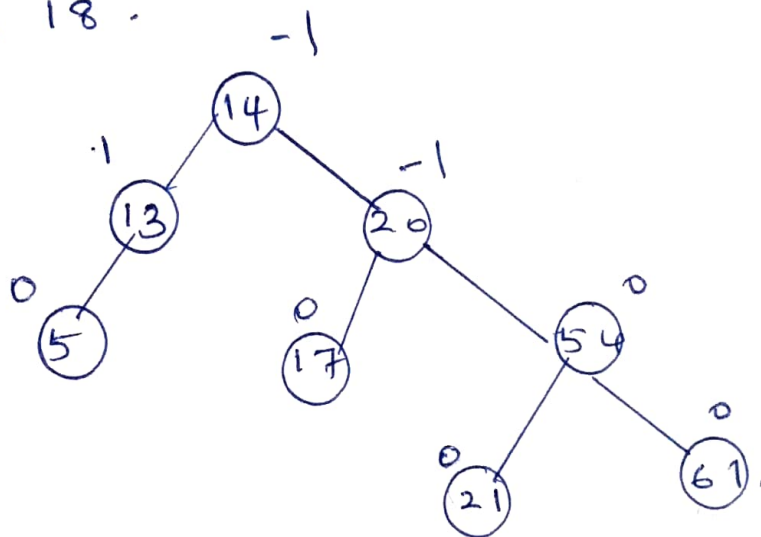
Either we replace 15 with its
inorder successor, i.e., 17

or by its inorder predecessor,
that is, 14.



Delete 18.

18 have only one ~~old~~ child, replace it with 18.



SETS ;

UNION & FIND OPERATIONS ON

DISJOINT SETS

An important application of the tree is the representation of sets.

A disjoint set data structure maintains a collection $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$ of disjoint dynamic sets.

Each set is identified by a representative, which is some member of the set.

If we have two sets S_x and S_y , $x \neq y$ such that $S_x = \{3, 4, 5, 6, 7\}$ and $S_y = \{1, 2\}$, then these sets are called disjoint sets as there is

no element which is common in both sets. (61)

Disjoint Set Operations

In dynamic set implementations, an object represents each element of a set.

Let 'x' denote an object.

Make-Set(x)

Creates a new set whose only member is x.

Union(x, y)

Unites the dynamic sets that contain x and y, say S_x and S_y into a new set that is the union of these two sets.

The two sets are assumed to be disjoint prior to the operation.

The representative of the resulting set is any member of $S_x \cup S_y$.

Since we require the sets in the collection to be disjoint, we destroy sets S_x and S_y , removing them from the collection.

Find-Set(x)

Returns a pointer to the representative of the set containing x .

Applications of Disjoint Set Data Structures

One of many applications of disjoint set data structures arises in determining the connected components of an undirected graph.

The procedure "Connected-Components" uses the disjoint set operations to compute the connected components of a graph.

Once Connected Components (63)

has been run as a preprocessing step, the procedure Same-Component answers queries about whether two vertices are in the same connected component.

```
Connected Components (G)
for each vertex  $v \in V(G)$ 
    do Make-Set ( $v$ )
for each edge  $(u, v) \in E(G)$ 
    do if Find-Set ( $u$ )  $\neq$  Find-Set ( $v$ )
        then Union ( $u, v$ )
```

```
Same-Component ( $u, v$ )
if Find-Set ( $u$ ) = Find-Set ( $v$ )
    then return TRUE
else
    return FALSE
```

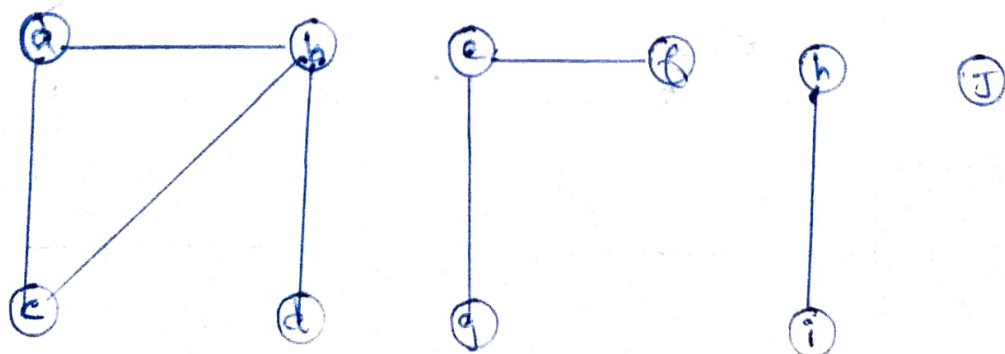
The procedure Connected-Component initially places each vertex v in its own set.

Then, for each edge (u, v) , it unites the sets containing u and v .

After all the edges are processed, two vertices are in the same connected component if and only if the corresponding objects are in the same set.

Thus Connected-Component computes sets in such a way that the procedure Same-Component can determine whether two vertices are in the same connected component.

Example.



Edge Processed	Collection of disjoint sets (6)								
Initial sets	$\{a\}$	$\{b\}$	$\{c\}$	$\{d\}$	$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$
(a,b)	$\{a,b\}$		$\{c\}$	$\{d\}$	$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$
(a,c)	$\{a,b,c\}$			$\{d\}$	$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$
(b,c)	$\{a,b,c\}$			$\{d\}$	$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$
(b,d)	$\{a,b,c,d\}$				$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$
(e,f)	$\{a,b,c,d\}$				$\{e,f\}$		$\{g\}$	$\{h\}$	$\{i\}$
(e,g)	$\{a,b,c,d\}$				$\{e,f,g\}$			$\{h\}$	$\{i\}$
(h,i)	$\{a,b,c,d\}$				$\{e,f,g\}$			$\{h,i\}$	

So the connected components are $\{a,b,c,d\}$, $\{e,f,g\}$ and $\{h,i\}$

Linked list Representation of disjoint sets.

A simple way to implement a disjoint set data structure to represent each set by its own linked list.

The object for each set has attributes head, pointing to the first object in the list, and tail, pointing to the last object.

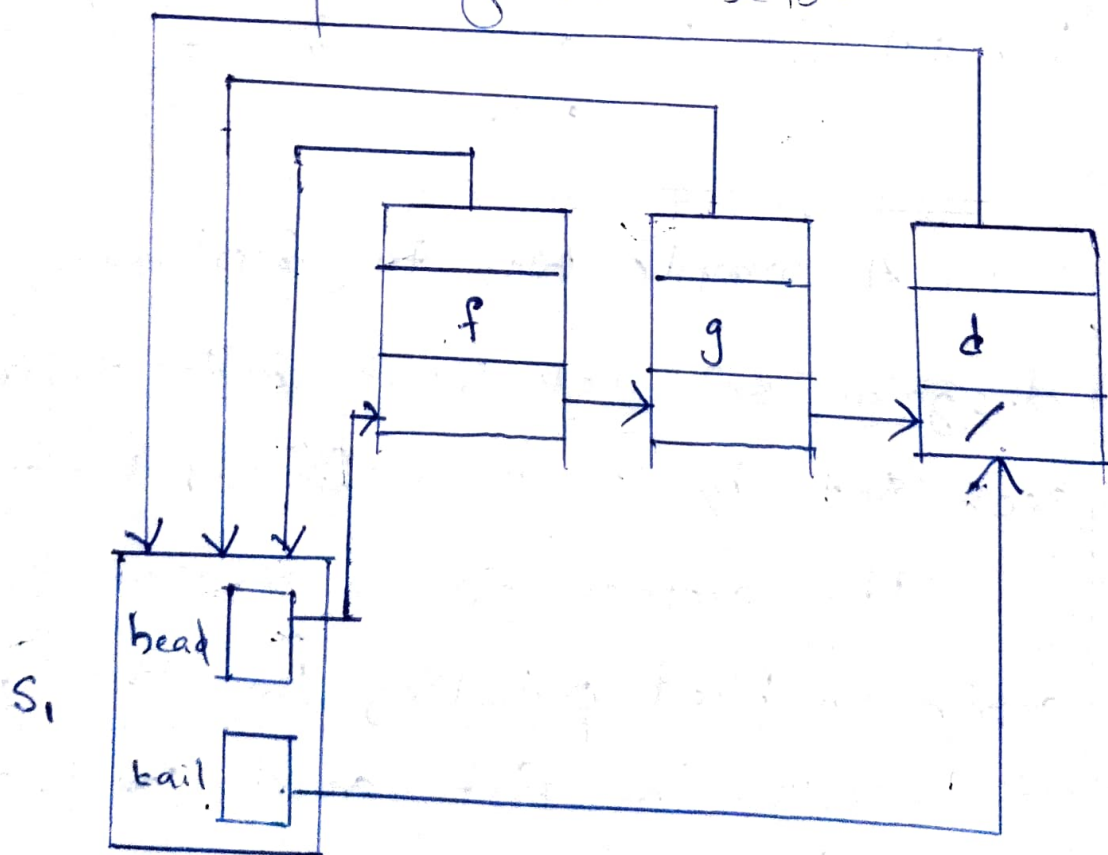
Each object in the list contains a set member, a pointer to the

next object in the list, and a pointer back to the Set object.

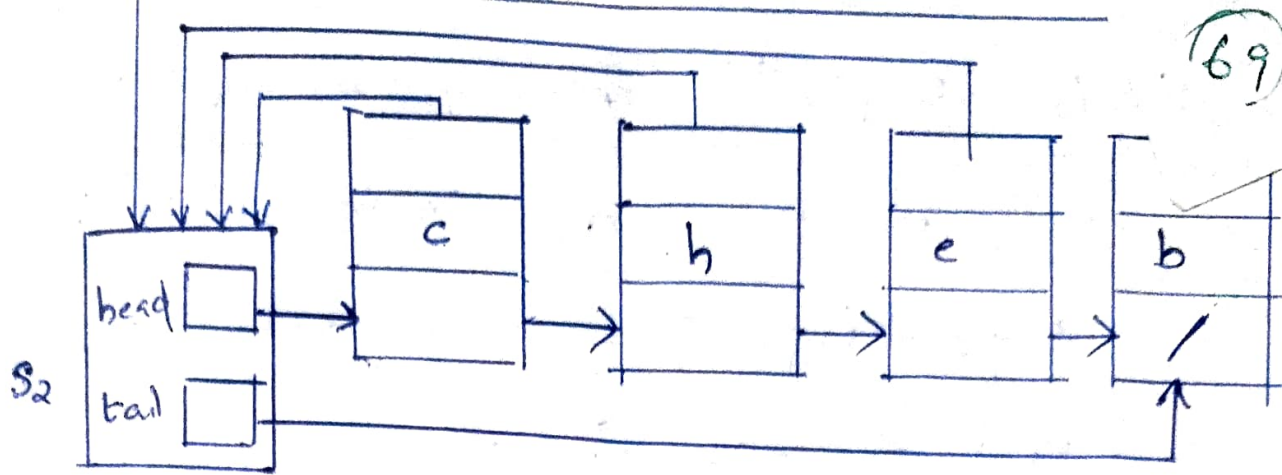
Within each linked list the objects may appear in any order.

The representative is the Set member in the first object in the list.

Figure below represents linked list repr. of two sets.



Set S_1 contains members d , f and g with representative f .



Set S_2 contains members b, c, e, and h with representative c.

With this linked list representation, both Make-Set and Find-Set are easy requiring $O(1)$ time.

To carry out Make-Set(x), we create a new linked list whose only object is x.

For Find-Set(x) we just follow the pointer from x back to its set object and then return the member in the object that head points to.

For example, the call Find-Set(g) would return f.

A Simple Implementation of Union

We perform Union (x, y) by appending y 's list onto the end of x 's list.

The representative of x 's list becomes the representative of the resulting set.

We use the tail pointer for x 's list to quickly find where to append y 's list.

Because all members of y 's list join x 's list, we can destroy the set object for y 's list.

We must update the pointer to the set object for each object on y 's list.

We execute the sequence of n Make-Set operations followed

by "n-1" Union operations.

(69)

Because the i^{th} Union operation updates i objects, the total no. of objects updated by all $n-1$ Union operations is $\sum_{i=1}^{n-1} i = O(n^2)$

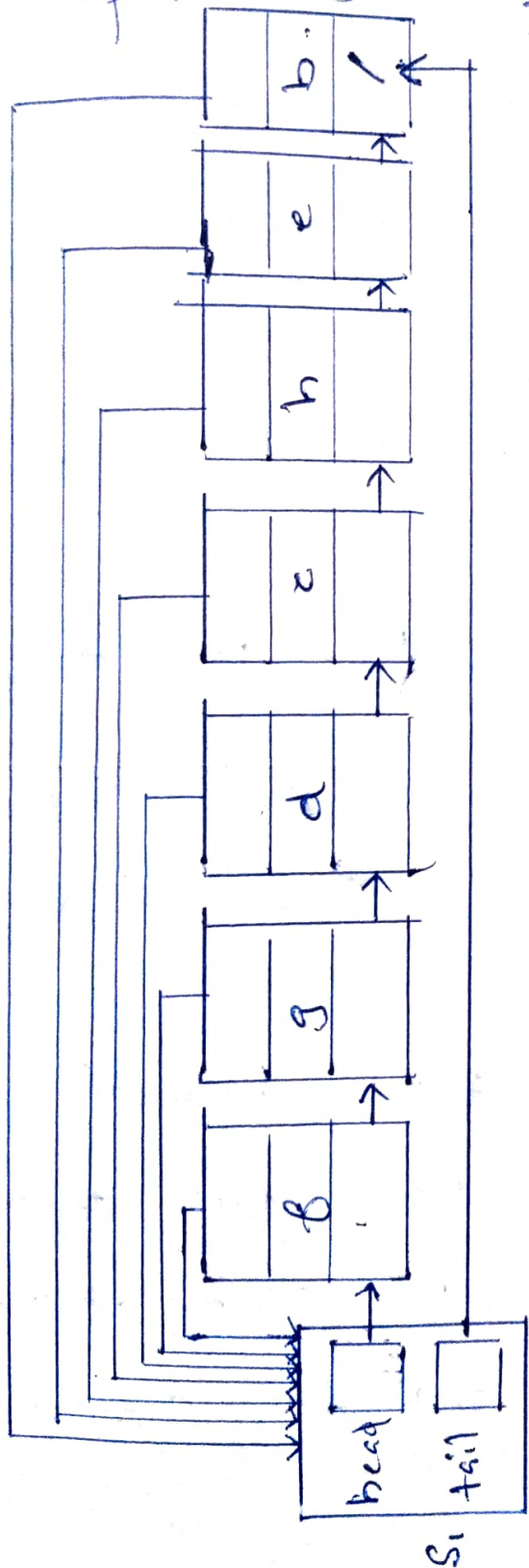


Fig:- The result of union (g,e) which

appends the linked list containing e to the linked list containing g.

The representative of the resulting set is g.

The set object for e's list, S_2 is destroyed.

Disjoint Set Forests

In a faster implementation of disjoint sets, we represent sets by rooted trees, with each node containing one member and each tree representing one set.

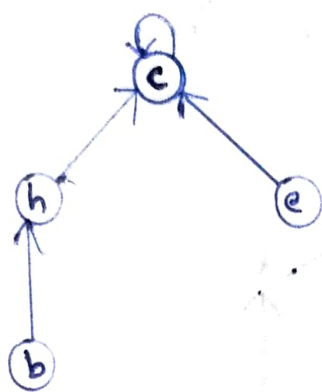
The root of each tree contains the representative and is its own parent.

The algorithms that use this representation are no faster than ones that use the linked list representation.

By introducing two heuristics using by rank and path compression we can achieve an asymptotically optimal disjoint set data structure.

$S = \{S_1, S_2\}$, $S_1 = \{c, b, e, b\}$ and

$S_2 = \{b, g, g\}$ can be represented as a disjoint set forest as shown below:



c is representative



b is representative

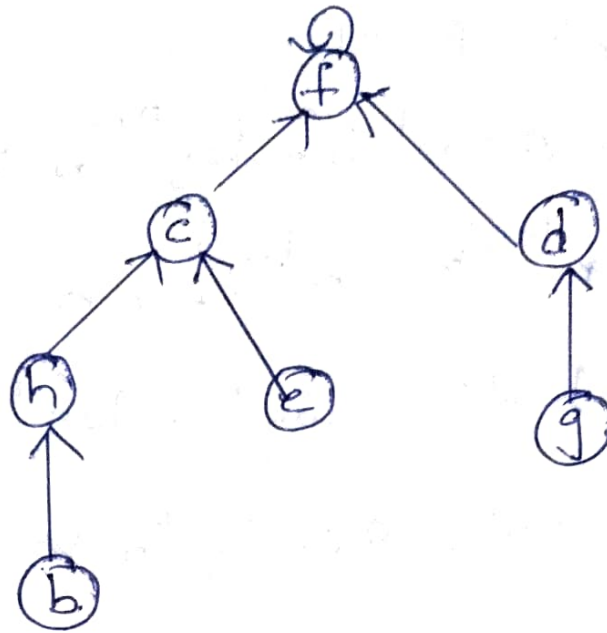
We perform the three disjoint set operations as follows:

A Make-Set operation simply creates a tree with just one node.

We perform Find-Set operation by following parent pointers until we find the root of the tree.

The nodes visited on this simple path toward the root constitute the find path.

Union operation causes the root of one tree to point to the root of the other.



Heuristics to improve the running time.

⇒ Union by rank [Weighted rule]

⇒ Path Compression [Collapsing rule].

The first heuristic Union by rank is similar to Weighted union heuristic.

This approach make the root of the tree with fewer nodes points to the root of the tree with more nodes.

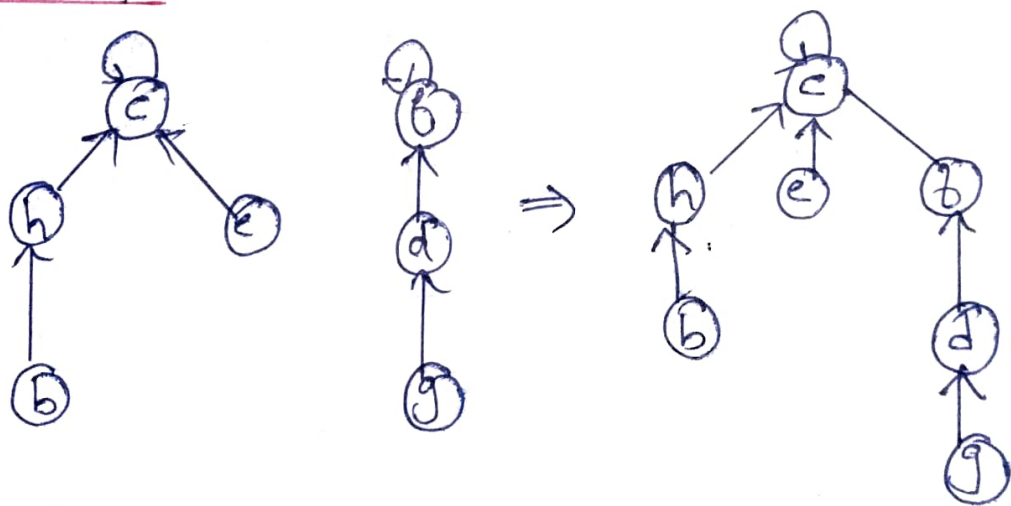
The second heuristic path compression
is also simple and highly effective.

Here, during find set operation, make each node on the find path point directly to the root.

Union by Rank [Weighted rule].

In union by rank, we make the root with smaller rank point to the root with larger rank with a union operation.

Example.



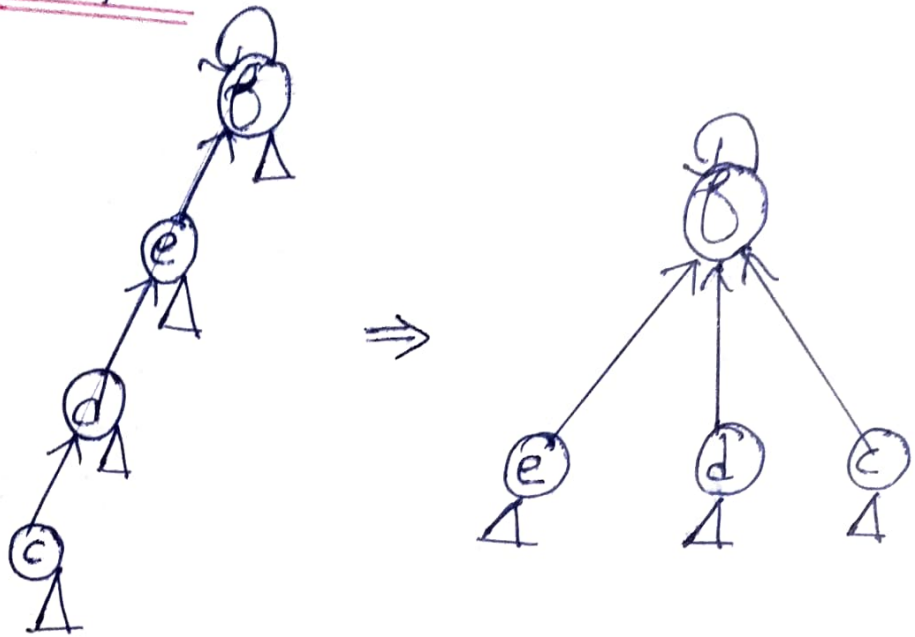
Path Compression - Collapsing rule.

This heuristic is used during findset operation to make each node on the

find path point directly to the root.

Path Compression does not change

any ranks.
Example.



Union by rank and path compression improves the running time of the operation on disjoint set forests. and improvement is even better when the two heuristics are used together.

When union by rank and path compressions are used together, the

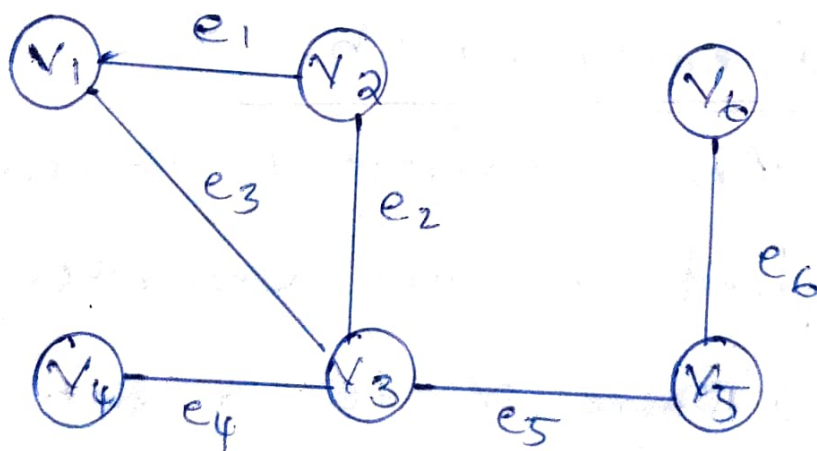
Worst case running time is $O(m, a(m))$
where $a(m)$ is a ~~very~~ very slowly
growing function.

Elementary Graph Algorithms

A Graph G consists of

- ① Set of vertices V [also called as nodes], and
- ② Set of edges E .

A Graph can be represented as $G(V, E)$, where, V is a finite non-empty set of vertices and E is a set of pair of vertices called edges.



$$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$$

$$E = \{e_1, e_2, e_3, e_4, e_5, e_6\}$$

$$= \{(v_1, v_2), (v_2, v_3), (v_1, v_3), (v_3, v_4), (v_3, v_5), (v_5, v_6)\}$$

Graph Traversal

A fundamental problem concerning graphs is the reachability problem.

Graph traversal means visiting all the nodes of the graph.

There are two graph traversal methods

⇒ Breadth First Search.

⇒ Depth First Search.

Breadth First Search.

In breadth first search, we start at a vertex v and mark it as having been visited.

The vertex v is at this time said to be unexplored.

A vertex is said to have been explored by an algorithm when the algorithm has visited all vertices adjacent to it.

All unvisited vertices adjacent from 'v' are visited next.

These are new unexplored vertices.

Vertex 'v' has now been explored.

The newly visited vertices haven't been explored and are put onto the end of a list of unexplored vertices.

The first vertex on this list is the next to be explored.

Exploration continues until no unexplored vertex is left.

The list of unexplored vertices operates as a queue.

Procedure

1. Input the vertices of the graph and its edges $G = (V, E)$.
2. Input the source vertex and

assign it to the variable s.

3. Insert the source vertex to the queue.

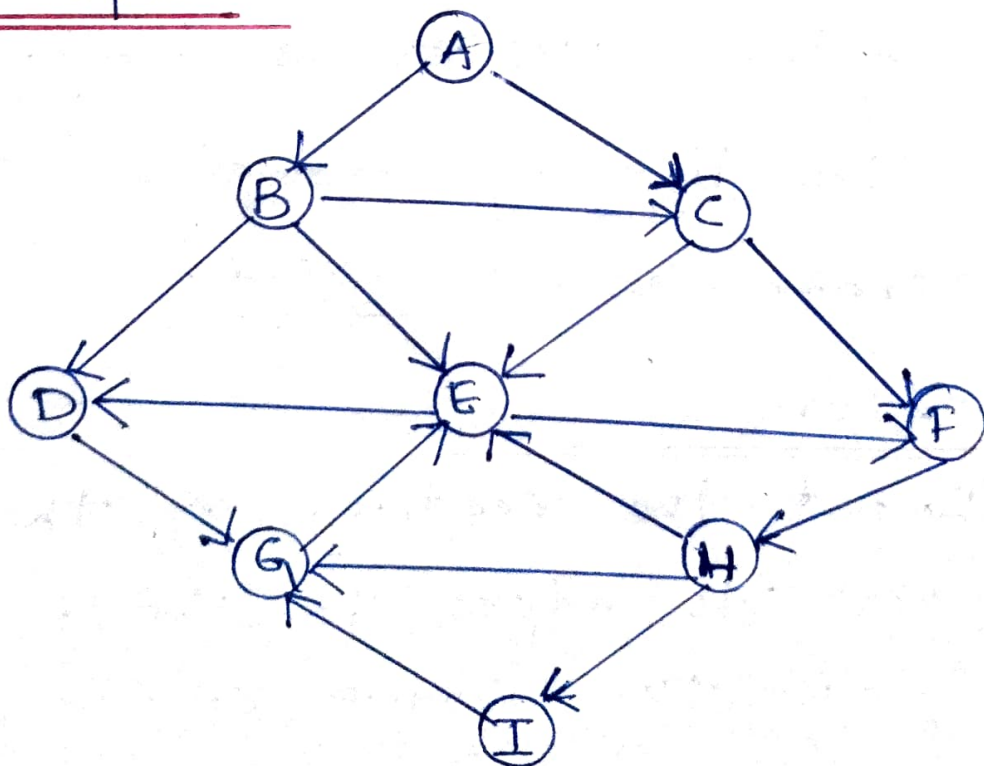
4. Repeat step 5 and 6 until the queue is empty.

5. Delete the front element of the queue and display it as visited.

6. Insert the vertices which are neighbour to the just deleted element, if it is not in the queue and not visited.

7. Stop.

Example.



Visited Node	Current Node	Queue List
	.	A
A	A	B C
AB	B	C D E
ABC	C	D E F
ABCD	D	E F G
ABCDE	E	F G
ABCDEF	F	G H
ABCDEFG	G	H
ABCDEFGH	H	I
ABCDEFGH I	I	No. neighbor nodes.
ABCDEFGH I *		

The nodes are visited in BFS
in the order
ABCDEFGHI

ALGORITHM

BFS(G, s)

{

$q = \text{queue}()$

$q.\text{enqueue}(s);$

~~visited[s] = 1~~

 While ($q \neq \text{empty}$)

 {

$v = q.\text{dequeue}();$

$x = v.\text{add}()$ // add v into

 visited [v] = 1

visited list
 x .

 for all u adjacent to v

 {

 if (u is not visited)

$q.\text{enqueue}(u);$

 }

 if $q.\text{empty}()$

 return x ;

Complexity

$O(V + E)$

V - No. of vertices

E - No. of edges

Depth First Search - DFS

The strategy followed by depth first search is, to search deeper in the graph whenever possible.

DFS explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it.

Once all of v 's edges have been explored, the search backtracks to explore edges leaving the vertex from which v was discovered.

This process continues until we have discovered all the vertices that are reachable from the original source vertex.

If any undiscovered vertices remain, then DFS selects one of them as a new source, and it repeats the search from that source.

The algorithm repeats this entire process until it has discovered every vertex.

Classification of edges.

We can define 4 edge types in terms of the depth first forest G_{π} produced by a depth first search on G :

1 $\Rightarrow$ Tree Edges

$\Rightarrow$ Edges in depth first forest G_{π}

$\Rightarrow$ Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .

2 $\Rightarrow$ Back Edges

$\Rightarrow$ Edges (u, v) connecting a vertex u to an ancestor v in a depth first tree.

$\Rightarrow$ Self-loops in directed graph also considered as back edges.

3 $\Rightarrow$ Forward Edges

$\Rightarrow$ Non tree edges (u, v) connecting a vertex u to a descendant v in a depth first tree.

4 $\Rightarrow$ Cross Edges.

$\Rightarrow$ All other edges.

$\Rightarrow$ They can go b/w vertices in the same depth first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth first trees.

When we first explore an edge (u, v) , the color of vertex v tells us something about the edge:

1 $\Rightarrow$ White indicate tree edge

2 $\Rightarrow$ Gray indicate a back edge and

3 $\Rightarrow$ Black indicate a forward or cross edge.

$\Downarrow$
if $u.d > v.d$

$\Downarrow$
if $u.d < v.d$

$$\underline{\underline{T(n)} = O(3^k)}$$

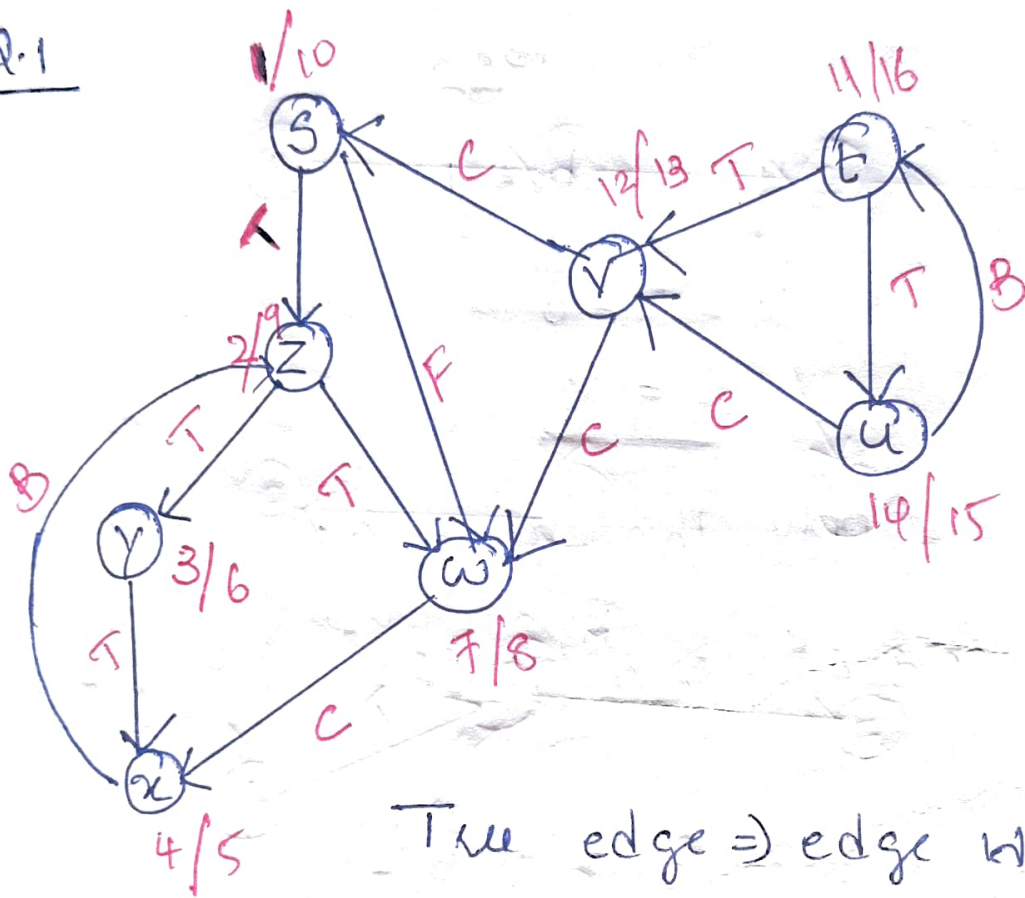
2.9 1.9

~~Time using Recursion tree method~~

$$\underline{\underline{T(n) = 3T(n/2) + O(n^2)}}$$

DFS - Edge classification

Q.1



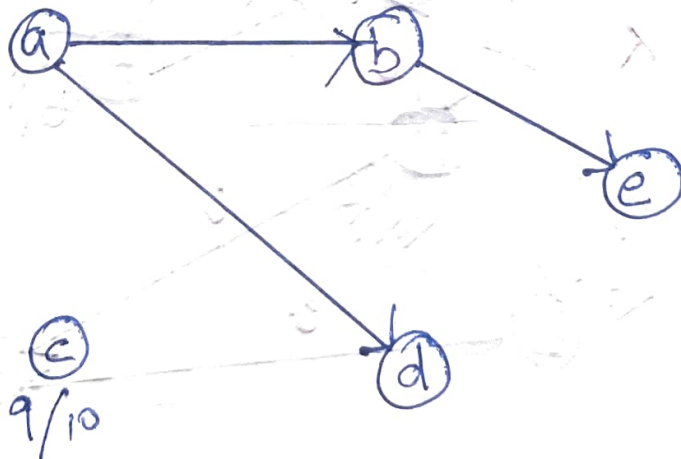
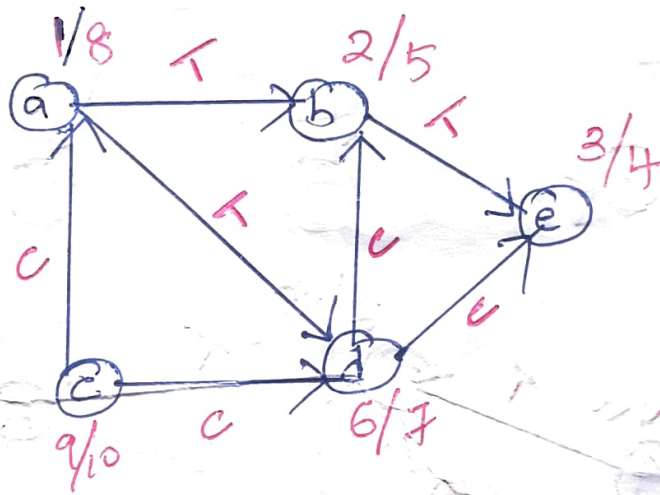
Tree edge $\Rightarrow$ edge which is included in DFS traversal.

Backward edge $\Rightarrow$ child $\rightarrow$ Parent
 Successor $\rightarrow$ Predecessor.

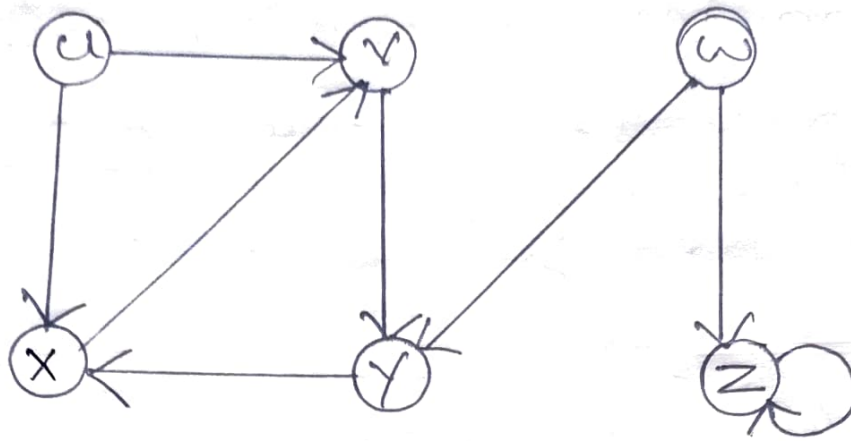
Forward edge $\Rightarrow$ Parent $\rightarrow$ child

Predecessor $\rightarrow$
Successor.

Cross edge $\Rightarrow$ node in two
different paths
or
tree.



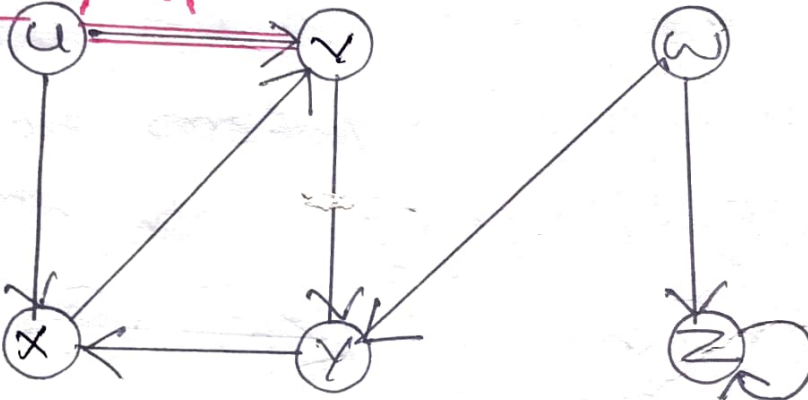
Example



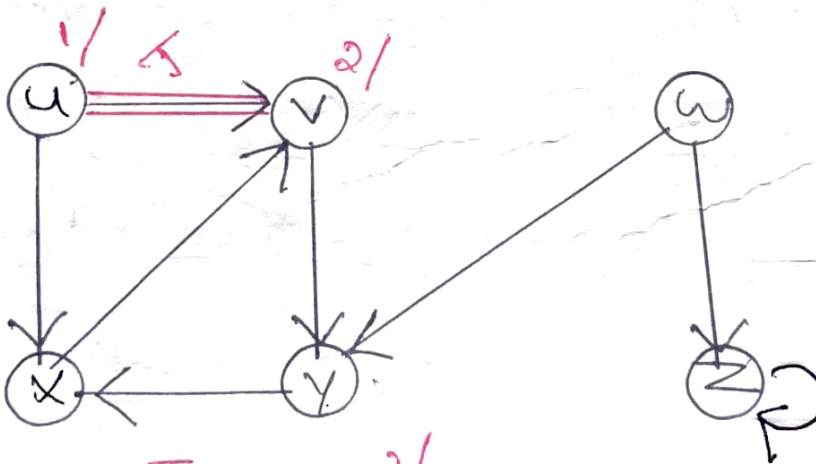
Let us start DFS from the

node u.

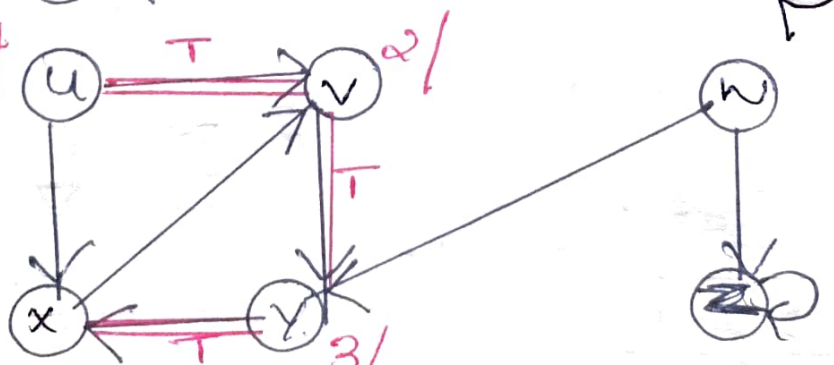
Node u

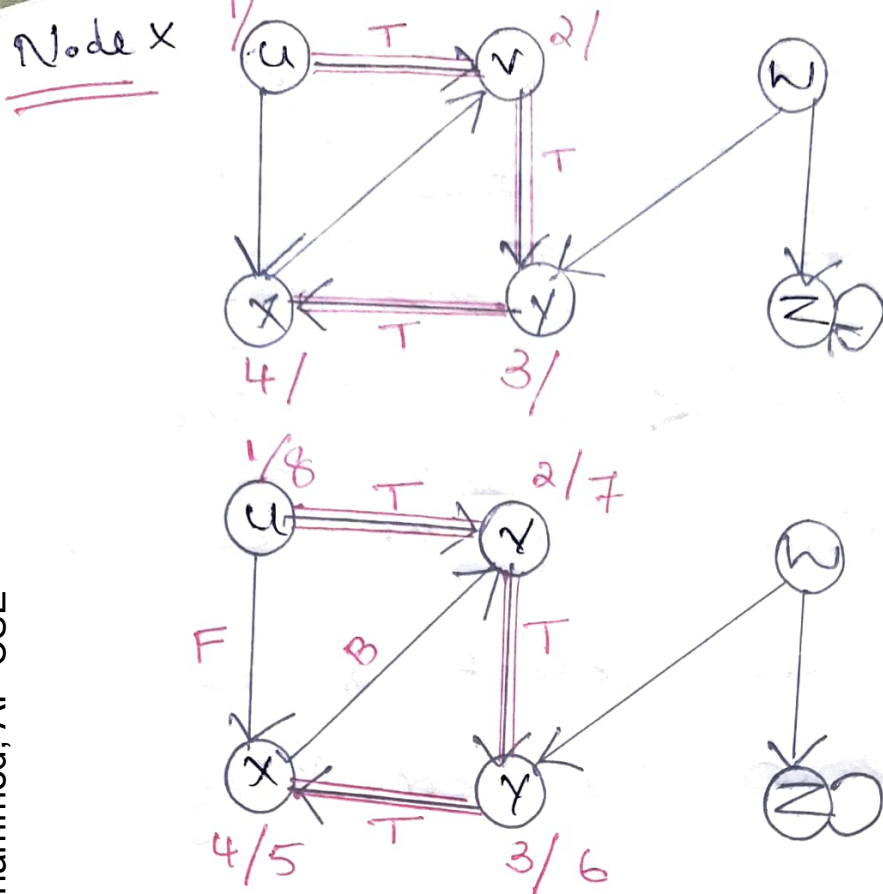


Node v



Node y





⇒ When node x is visited, its neighbour node ~~is~~ v , which is already visited.

⇒ So we explore the node x , and the edge (x, v) becomes a non tree edge ⇒ Back edge

⇒ It becomes back edge because at the time of visiting x the color of ~~edge~~ node v is gray.

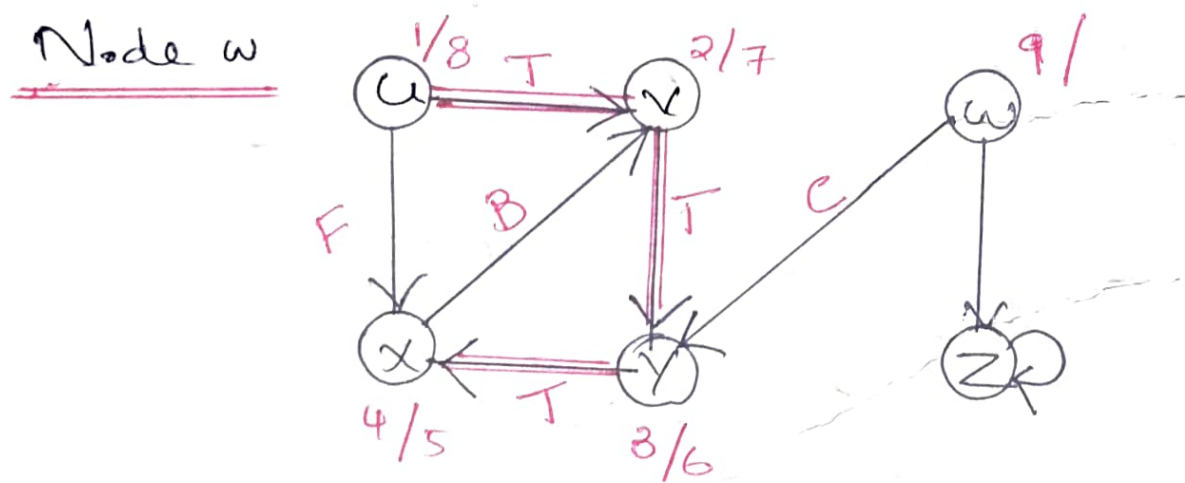
⇒ And then we backtrack through the path

⇒ We explore node y, v and u .

⇒ When exploring the edge 'u', color of the node x is black and considering the edge (u, x), $u.d < x.d$ so the edge becomes forward edge.

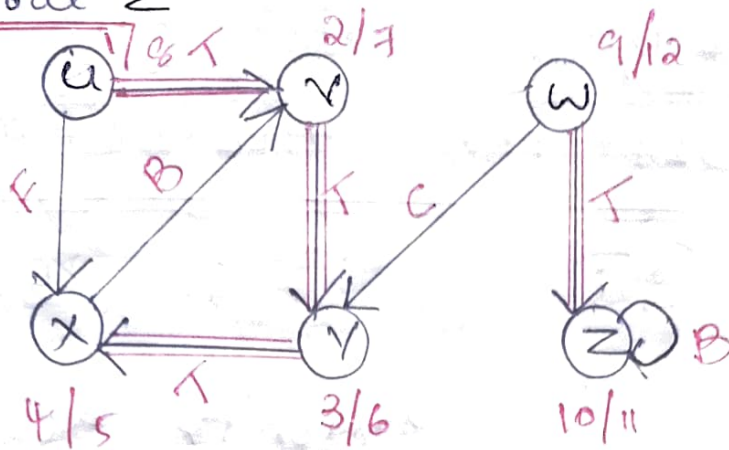
⇒ Then we select a new source from the remaining nodes.

⇒ Let us take w as the new source.



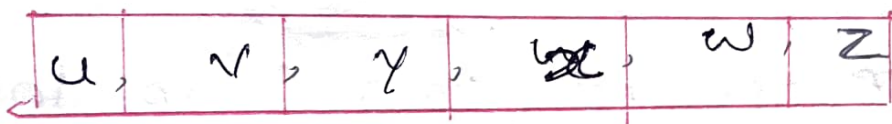
⇒ When traversing node 'w', the edge (w, y), y is already explored and its color is black and since $u.w > u.y$ it is cross edge.

Node z



⇒ Self loops are considered as back edge.

⇒ So here BFS is performed in the order:-



Analysis

⇒ For a graph $G(V, E)$, the algorithm is called for each vertex $v \in V$, and for each vertex, it takes $O(E)$ times to find out $|adj(v)|$.

⇒ Therefore total running time for the algorithm is $O(V + E)$

Algorithm

DFS (G)

for each vertex $u \in V(G)$

$u.color = white$

$u.\pi = Nil$

$time = 0$

for each vertex $u \in V(G)$

if $u.color == white$

DFS-Visit (G, u).

DFS-Visit (G, u)

$time = time + 1$

$u.d = time.$

$u.color = Grey$

for each v adjacent to u

if $v.color == white$

$v.\pi = u$

DFS-Visit (G, v)

$u.color = Black.$

$time = time + 1$

$u.f = time.$

Complexity

For a graph $G(V, E)$, the algorithm is called for each vertex $v \in V$ and for each vertex, it takes $O(E)$ times to find adjacent (v) .

Therefore total running time for the algorithm is

$$\underline{\underline{O(V + E)}}$$

Topological Sorting

A topological sort of a dag $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge (u, v) , then u appears before v in the ordering.

A topological sort of a graph can be viewed as an ordering of its vertices along a horizontal line so that all directed edges go from left to right.

DAG $\Rightarrow$ Directed Acyclic Graph

Topological sort is an application of DFS.

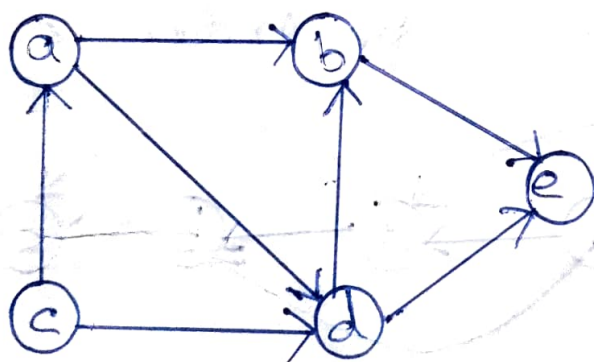
The following simple algorithm topologically sorts a dag.

Algorithm

Topological Sort (G)

call DFS(G) to compute finishing times for each vertex 'v'
as each vertex is finished,
insert it onto the front of a
linked list.
return the linked list of vertices

Example

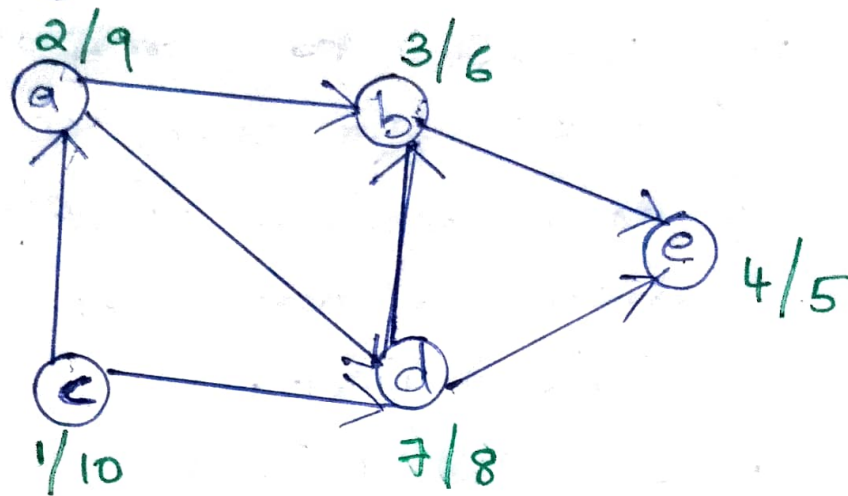


⇒ DFS should start at a vertex,
whose in degree = 0.

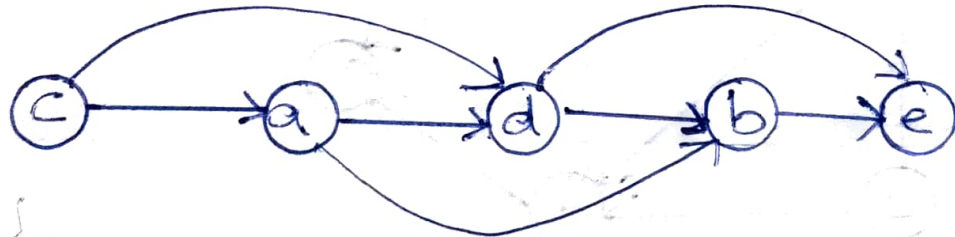
⇒ in degree of a vertex is 0 when
there is no edges incident on
it.

⇒ Here 'c' is the vertex with
in degree 0.

⇒ So we start the DFS by taking 'c' as the source vertex.



Arrange the nodes horizontally in decreasing order.



Running time of topological sorting is $O(V+E)$.

Strongly Connected Components.

A directed graph is strongly connected if every two vertices are reachable from each other.

The strongly connected components of a directed graph are the equivalence classes of vertices under the "mutually reachable" relation.

Decomposing a directed graph into its strongly connected components is a classic application of depth first search.

Procedure

- ① Find out the transpose of the input graph $G(V, E)$.
- ② Perform DFS on G^T .
- ③ Perform DFS on G in the decreasing order of DFS performed on G^T .

Algorithm.

Strongly Connected Components (SCC)

Call DFS (G)

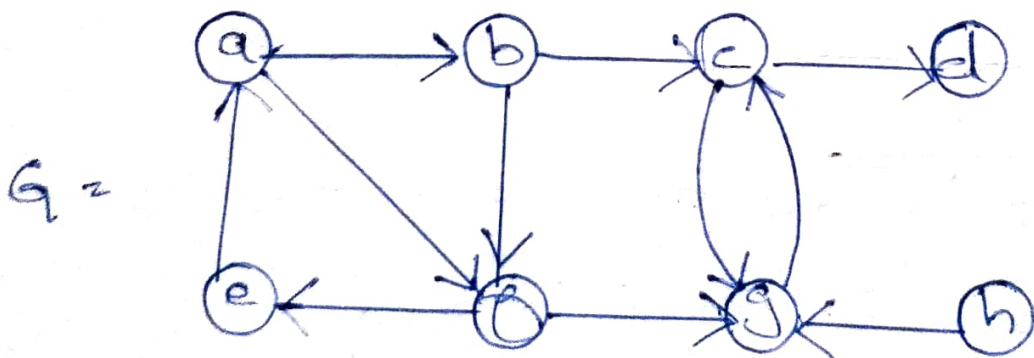
Compute G^T

Call DFS (G^T) to compute finishing times $f(u)$ for each vertex u .

Call DFS (G), in order of decreasing $f(u)$.

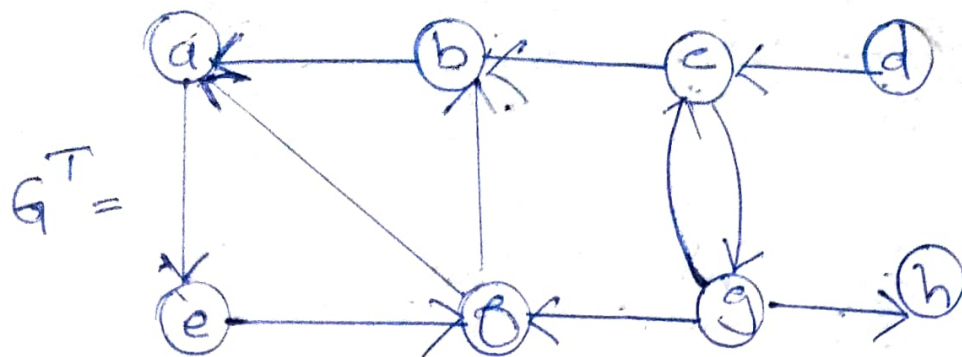
Output the vertices of each tree in depth first forest as a separate strongly connected component.

Example

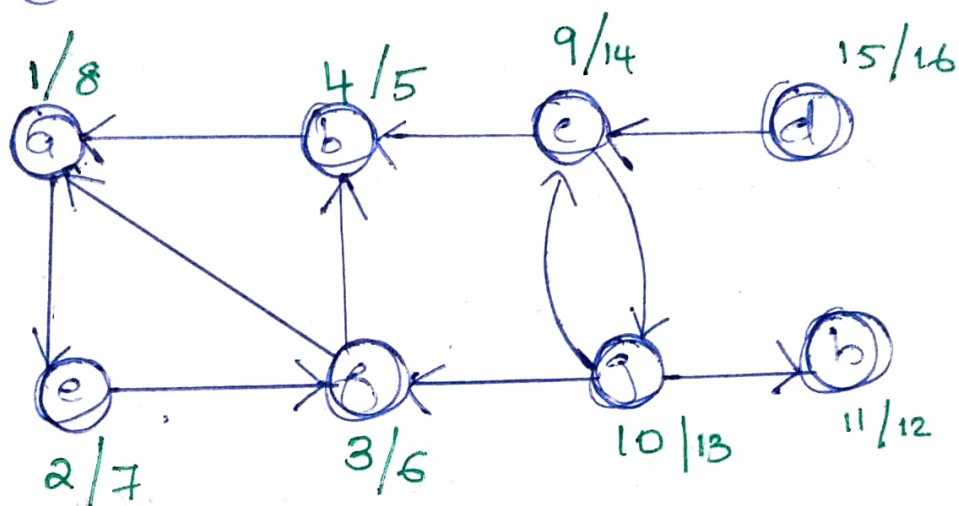


We have to find out the transpose of this graph.

①



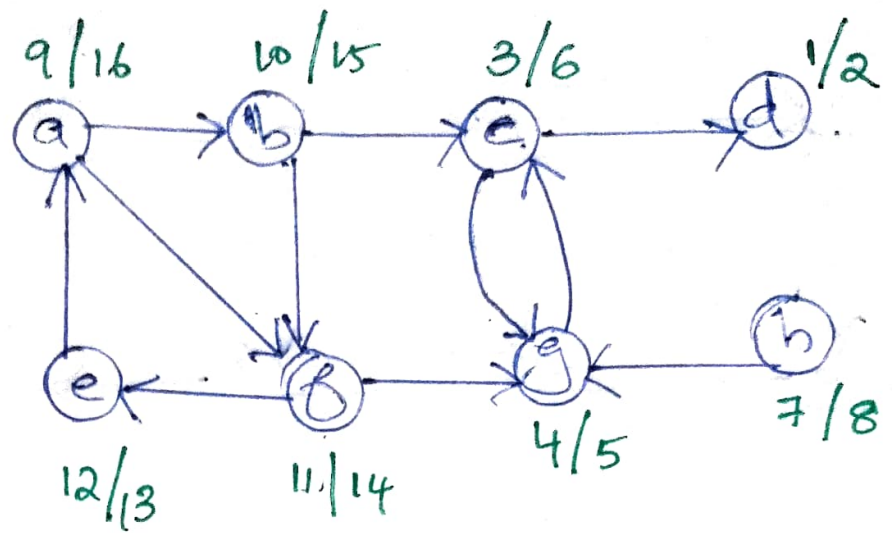
② Perform DFS on it.



Decreasing order of finishing time is

d, c, g, h, a, e, f, b

③ Perform DFS on the original graph in the decreasing order of finishing time as shown above.



The strongly connected components are
 $\{a, b, e, f\}$ $\{c, g\}$

So we can redraw the graph as

